Table des matières

Stratégie de compteurs et durcissement du contrat FDM 1
1. Stratégie : par type d'événement, mono-terminal . . . 1
2. JSON canonique sur le câble . . . 2
3. Clé d'idempotence . . . 2
4. Blocage opérationnel sur déconnexion . . . 3
5. Compteur sans trou - fix d'intégrité (B3) . . . 3
6. Reproductibilité . . . 4

Stratégie de compteurs et durcissement du contrat FDM

Date : 2026-05-14 Périmètre : Stratégie de numérotation séquentielle utilisée par le POS, mécanisme de déduplication par le FDM, et forme du payload sur le câble.

Le Manuel de certification SCE 2.0 exige du POS qu'il (a) déclare sa stratégie de compteurs dans le formulaire de demande de certification et (b) démontre qu'il ne peut pas produire de trou dans la séquence en exploitation normale ni récupérer de manière incorrecte d'une erreur FDM. Le présent document est la trace officielle des deux.

---

1. Stratégie : par type d'événement, mono-terminal

La spec POS v2.2 §385-388 autorise quatre stratégies :

1. Distincte par type d'événement.
2. Continue générale tous types confondus.
3. Continue générale par terminal.
4. Distincte par type d'événement, par terminal.

Nous utilisons la #1 - distincte par type d'événement. Le MVP étant mono-terminal, le qualificatif "par terminal" est sans objet aujourd'hui ; si un mode multi-terminal est ajouté ultérieurement, la stratégie est rétro-compatible parce que la clé d'idempotence FDM (cf. §3) discrimine déjà par `terminalId`.

Pourquoi cette stratégie

— Traçabilité maximale. Un saut sur le compteur N est significatif ("une vente a été manquée") et isolé de l'activité S/F/P/R. Avec la stratégie #2, un événement non-fiscal (ouverture tiroir par exemple) et une vente partagent la même séquence monotone, brouillant la piste d'audit.
— Wrap sûr. Les compteurs par label se bouclent à 999 999 999 (max POS v2.2). Aux volumes Tokyo Sushi Bar (~150 tickets/jour), le compteur N mettrait ~18 000 ans à boucler. Les autres labels sont bornés similairement.
— Récupération indépendante. Un bug de gestion sur un label (par exemple un flux de remboursement défectueux corrompant C) ne peut pas se propager à un autre label.

Implémentation

`crates/pos-fiscal/src/counter.rs::FiscalCounter::next(conn, label)` implémente un UPSERT SQLite transactionnel sur la table `fiscal_counters` :

```
CREATE TABLE IF NOT EXISTS fiscal_counters (
    event_label TEXT PRIMARY KEY,
    current_value INTEGER NOT NULL DEFAULT 0
);

INSERT INTO fiscal_counters (event_label, current_value)
VALUES (?1, 1)
ON CONFLICT(event_label)
```

```
DO UPDATE SET current_value =
    CASE WHEN current_value >= 999999999 THEN 1
         ELSE current_value + 1
    END;
```

Huit tests unitaires dans `counter.rs` couvrent : démarrage à 1, incrément monotone, indépendance par type d'événement, `current()` reflète le dernier `next()`, sémantique table vide, et - depuis le fix B3 - l'invariant peek/next.

Cette stratégie est verrouillée et NE DOIT PAS changer post-certification. Au titre de l'article 109 de la procédure de certification, un changement de stratégie nécessite un nouveau cycle de soumission.

---

2. JSON canonique sur le câble

Module : `crates/pos-fiscal/src/canonical.rs`. Utilisé par : `fdm_client::FdmClient::execute_raw` pour chaque corps de requête sortant.

Règles :

- Clés d'objet triées par codepoint Unicode à chaque niveau.
- Aucun espace insignifiant.
- Normalisation `serde_json` par défaut des nombres (forme la plus courte).
- Ordre des tableaux préservé (les séquences sont sémantiques).

Pourquoi envoyer du JSON canonique

Le FDM effectue sa propre canonicalisation pour la signature, donc notre choix de sérialisation N'AFFECTE PAS la signature numérique qu'il retourne. Nous canonicalisons néanmoins parce que :

1. Octets de requête déterministes - deux encaissements avec des entrées identiques produisent des corps de requête bit-à-bit identiques, ce qui rend les diff de journaux propres.
2. Débogage de la clé d'idempotence - quand un doublon est détecté, la spec compare champ par champ, mais disposer d'octets canoniques reproductibles permet de calculer et journaliser un hash de requête stable pour recoupement.
3. Pérennité - si une table locale de retry par hash de requête est introduite plus tard, `canonical::to_canonical` est la primitive correcte.

Vérification

8 tests unitaires dans `canonical.rs::tests` couvrent :

Propriété	Test
Clés top-level triées	`canonical_sorts_top_level_keys`
Invariance à l'ordre des champs	`canonical_is_field_order_invariant`
Clés d'objets imbriqués triées	`canonical_sorts_nested_objects`
Ordre des tableaux préservé	`canonical_preserves_array_order`
Déterministe sur appels répétés	`canonical_is_deterministic`
Pas d'espace dans la sortie	`canonical_no_whitespace_in_output`
Clés Unicode sans panique	`canonical_handles_unicode_keys`
Test propriété 50 inputs pseudo-random	`canonical_property_random_inputs_deterministic`

---

3. Clé d'idempotence

Type : `crates/pos-fiscal/src/inputs.rs::IdempotencyKey<'a>`.

Selon la spec FDM v1.4 §3, le FDM déduplique sur le quintuplet :

`{ posId, posDateTime, terminalId, eventLabel, posFiscalTicketNo }`

- Quintuplet identique, payload identique : le FDM renvoie la réponse mise en cache et n'avance pas ses compteurs internes. Réessai sûr.
- Quintuplet identique, payload différent : le FDM refuse (réponse d'erreur). Le POS ne doit PAS écraser silencieusement - c'est la branche "conflit" de la spec.
- Quintuplet différent : le FDM traite comme un nouvel événement et signe.

Nous fournissons une méthode sur chaque input spécifique d'événement (`SaleInput::idempotency_key()`, `OrderInput::idempotency_key()`, ⋯) pour que les appelants n'aient pas à se souvenir du mapping label correspond à input. Cinq tests unitaires (`inputs.rs::idempotency_tests`) couvrent le contrat d'égalité et le mapping de label par type.

La clé expose aussi `render()` pour des lignes de log grep-friendly : `posId=CPOS01,posDt=2026-05-14T10:00:00+02:00,terminalId=T1,label=N,ticketNo=42`.

---

4. Blocage opérationnel sur déconnexion

Implémenté dans `src-tauri/src/commands/checkout.rs::do_checkout` : quand `FdmClient::sign_sale` retourne `FdmError::NotConnected`, la commande d'encaissement renvoie `success: false` avec un message d'erreur côté utilisateur et ne valide pas la commande (la transaction SQLite est avortée, aucun compteur n'avance - voir §5).

La feature Cargo dev-only `allow-offline` peut contourner cela en développement local - elle n'est pas incluse dans la build de certification (`cargo build --release --no-default-features --features cert`).

Reporté : sondage des statuts dégradés

Aujourd'hui, nous ne bloquons que sur les échecs de connexion TCP-niveau du FDM. Le FDM expose aussi des drapeaux de dégradation opérationnelle via la query `status` (`errors[]`, `warnings[]`, `bufferCapacityUsed`). Un travail futur consistera à lancer un poller tokio en arrière-plan qui miroire ces flags dans un `Arc<RwLock<FdmOperationalStatus>>`, et à faire bloquer également la commande d'encaissement quand le FDM est joignable mais signale des erreurs critiques. La spec n'exige strictement le blocage que sur déconnexion dure, ce que nous faisons déjà.

---

5. Compteur sans trou - fix d'intégrité (B3)

Problème historique : dans une version antérieure du code, `FiscalCounter::next()` était appelé avant la mutation FDM. Si l'appel FDM échouait (erreur non-`NotConnected`, timeout, ou crash en cours), le compteur local avait déjà avancé et l'encaissement suivant utilisait un numéro plus grand - produisant un trou dans la séquence locale, en violation de l'exigence "pas de trou".

Correctif appliqué (B3, 2026-05-14) :

1. Nouvelle méthode **`FiscalCounter::peek_next`** : retourne la valeur qu'aurait renvoyée `next()` sans muter la table - pure lecture (`current() + 1` avec gestion du wrap à 999 999 999).
2. Refonte du flow de toutes les commandes mutant un événement fiscal :
   - Étape 1 : `peek_next` pour réserver le numéro de ticket à envoyer au FDM (aucune mutation DB).
   - Étape 2 : envoi de la requête au FDM (potentiellement échouante).
   - Étape 3 : sur succès UNIQUEMENT, à l'intérieur d'une transaction SQLite `BEGIN IMMEDIATE` atomique :
     - `FiscalCounter::next` (commit de l'incrément),
     - `archive::archive_event` (écriture de la ligne fiscale),
     - fermeture de la commande / autre side-effect.

— Étape 4 : sur échec, retour immédiat sans aucune mutation - compteur intact, prochain essai utilise la même valeur.

Si l'opérateur retente, le quintuplet d'idempotence est identique (même `posFiscalTicketNo`), le FDM renvoie la réponse mise en cache et la séquence reste sans trou.

3. Tests de régression dédiés dans `src-tauri/tests/spf_e2e_no_gap.rs` :

— `checkout_fdm_failure_does_not_advance_counter` : FDM renvoie 500, le compteur N reste à 0, aucune ligne archivée, commande toujours ouverte.
— `refund_fdm_failure_does_not_advance_counter` : après une vente réussie (N=1), un remboursement avec FDM 500 ne touche pas le compteur N (reste à 1) et n'archive aucune ligne supplémentaire.

Les commandes touchées par le refactor : `do_checkout`, `do_refund_checkout`. Les autres commandes (`do_sign_table_order`, `do_sign_pre_bill`, `do_money_in_out`, `do_open_drawer_fiscal`, `do_work_in/out`, `do_copy_ticket`, `do_generate_x_report`, `do_generate_z_report`) utilisent un pattern différent - `.ok()` sur l'erreur FDM avec archive systématique avec `sign_result=None` - qui n'a pas le bug parce que compteur et archive sont déjà alloués ensemble. Voir aussi `DECLARATION_FONCTIONS_INTERDITES_FR.md` §4.

---

6. Reproductibilité

```
cd moyun-pos
cargo test --package pos-fiscal --lib counter
cargo test --package pos-fiscal --lib canonical
cargo test --package pos-fiscal --lib idempotency_tests
cargo test --package moyun-pos --test spf_e2e_no_gap
```

Totaux workspace à la dernière révision : 190 tests passants, 0 échec, 2 ignorés (rehearsal staging).